Introduction to Deep Learning

Prof. Olushina Olawale Awe

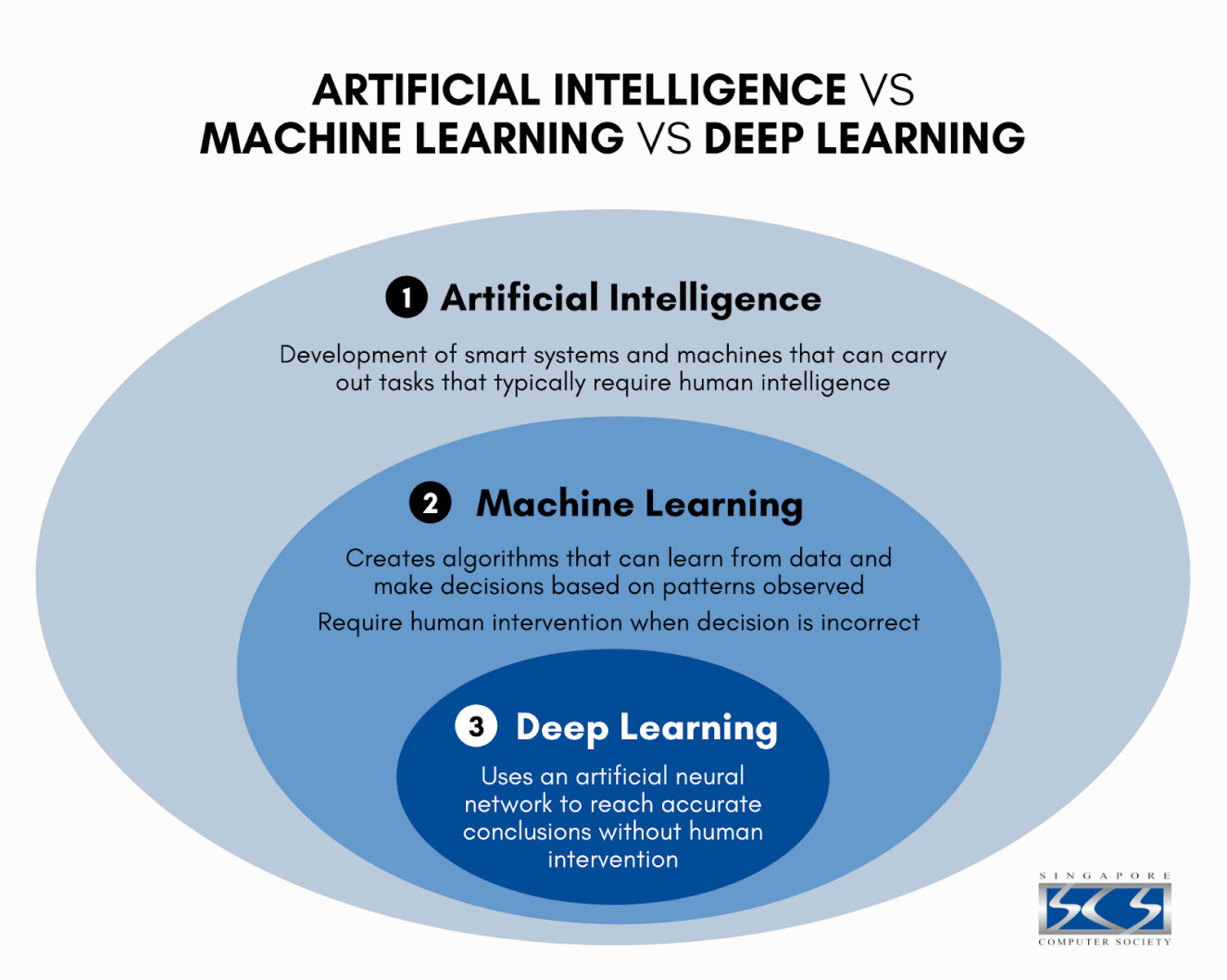
PH Ludwigsburg University, Germany

December 4, 2025

Outline

1. Background
2. Feedforward Neural Networks (FFNNs)
3. Deep Neural Networks
4. Core Training Concepts
5. Gradient Descent Questions
   - Tasks: Single-Multilayer-Hidden-Neuron Network
6. Implementation and Practice
7. 2. Forward Pass
8. 3. Backpropagation
9. 4. Demonstration: Practical Datasets
10. Conclusion
11. Practical Demo (PyTorch)
12. Exercises

AI, ML, and DL Hierarchy



AI (broad goal) $\supset$ **Machine Learning** (data-driven) $\supset$ **Deep Learning** (neural networks)

What is Deep Learning?

- A subfield of Machine Learning focused on learning complex patterns using **multi-layer neural networks**.
- Inspired by the structure and function of the human brain.
- Learns hierarchical representations from raw data (images, text, audio).
- Powers modern AI: GPT, BERT, self-driving cars, medical AI, robotics.

Why Deep Learning Matters Today

- Dominates state-of-the-art in NLP, speech, and reinforcement learning.
- Enables automatic feature extraction.
- Scales with data and computation.
- Forms the foundation of large foundation models (LLMs, Vision Transformers).
- DL models have achieved human-level or superhuman performance in several domains.

Applications of Deep Learning

- Medical imaging and diagnostics
- Fraud detection and finance
- Climate modeling and remote sensing
- Natural language processing (GPT, BERT)
- Civic applications like governance dashboards, citizen sentiment analysis

History and Milestones

- 1943 – McCulloch & Pitts neuron
- 1958 – Rosenblatt's Perceptron
- 1986 – Backpropagation (Rumelhart et al.)
- 2012 – AlexNet revolution in vision
- 2017 – Transformers for sequence modeling
- 2020s – Foundation models and LLMs

Deep Learning vs. Traditional Machine Learning

Aspect	**Traditional Machine Learning**	**Deep Learning**
Feature Engineering	Requires hand-crafted features designed by experts	Learns hierarchical features automatically from raw data
Data Requirements	Works well with small–medium sized structured datasets	Typically needs very large datasets, especially for vision/NLP
Model Complexity	Simpler models (e.g., linear/logistic regression, SVM, trees)	Very high-capacity models (deep neural networks with many layers)
Computational Cost	Can run on standard CPUs; training usually fast	Often requires GPUs/TPUs; training can be computationally expensive
Interpretability	Often more interpretable (e.g., linear models, shallow trees)	Frequently "black-box"; requires XAI methods (SHAP, LIME, etc.)
Typical Data Types	Mainly structured/tabular data, simple time series	Images, text, audio, video, complex high-dimensional

Mathematical Perspective

- Goal: approximate a complex function $f^*(x)$.
- Neural networks define $f(x;\theta)$ and learn parameters θ minimizing loss $L(y,\hat{y})$.
- Universal approximation theorem:

$$\forall f^*, \exists\ f(x;\theta) \text{ s.t. } |f(x;\theta) - f^*(x)| < \epsilon$$

The Artificial Neuron (Perceptron)

A perceptron is a *simple neuron*. A multilayer neural network is a *more powerful, differentiable perceptron* used for deep learning.
The perceptron models a single decision unit:

$$y = f(\mathbf{w}^T x + b)$$

- x: input vector
- w: weights
- b: bias
- f: activation function

What is the Bias in a Neuron?

A typical neuron computes:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

- w_i are the **weights**.
- b is the **bias term**.
- The activation is then: $a = \sigma(z)$.

Intuition: The bias shifts the input to the activation function, giving the neuron more flexibility.

Purpose of Bias in Neural Networks

- **Shifts the activation:**
  Bias allows the neuron to move the activation function left or right.

$$z = w^\top x + b$$

  Without b, the neuron output is constrained to pass through the origin.

- **Learns the intercept:**
  Analogous to the intercept term in linear regression:

$$y = wx + b$$

- **Increases model flexibility:**
  Enables the network to represent functions that are not forced to be zero at $x = 0$.

Bias = small parameter but has huge impact on model capacity.

From Single Neuron to Neural Networks

- A neural network stacks many neurons in layers.
- Information flows from input $\rightarrow$ hidden $\rightarrow$ output.
- Deeper layers learn more abstract features.

$$y = f_L(W_L f_{L-1}(\cdots f_1(W_1 x)))$$

Hierarchical Feature Learning

Example: Image Processing

- Layer 1: Edges
- Layer 2: Shapes
- Layer 3: Object parts
- Layer 4: Objects (faces, cars)

Deep learning learns these automatically.

Mathematical Deep Dive: Perceptron vs. Modern Neuron

1. Perceptron Model (Rosenblatt, 1957)

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$\hat{y} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

Characteristics

- Uses a **Heaviside step function** $H(z)$.
- Output is strictly binary.
- **Not differentiable** $\rightarrow$ cannot use gradient descent.
- Can only learn **linearly separable** decision boundaries.

2. Modern Artificial Neuron (Deep Learning)

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$\hat{y} = \sigma(z)$$

Where $\sigma(z)$ is a differentiable activation function:

Weights, Loss, and Learning

- **Weights**: parameters the network learns.
- **Loss function**: measures error.

$$L = (y - \hat{y})^2 \quad \text{(example)}$$

- **Gradient descent**: updates weights to reduce loss.
- **Backpropagation**: computes gradients using chain rule.

Gradient Descent Intuition

Move weights in the direction that decreases error:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

- η: learning rate
- The network learns by adjusting weights over many iterations.

Learning Rate in Deep Learning

Definition

- The **learning rate** (denoted η) is a hyperparameter that controls how large the weight updates are during gradient descent.

Mathematical Role

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial \mathcal{L}}{\partial w}$$

Interpretation

- η determines how **fast or slow** a model learns.
- Too large η: updates overshoot $\rightarrow$ training may **diverge**.
- Too small η: updates are tiny $\rightarrow$ training becomes **very slow**.

Why It Matters

- A good learning rate accelerates convergence.
- It stabilizes training and prevents oscillations.

Key Insight: The learning rate is NOT an activation function. It is a **training hyperparameter** that controls gradient descent behavior.

Structure of FFNN

- Input layer: raw features
- Hidden layers: nonlinear transformations
- Output layer: predictions (class scores, regression output)

$$\hat{y} = f(x) = W_3\sigma(W_2\sigma(W_1x + b_1) + b_2) + b_3$$

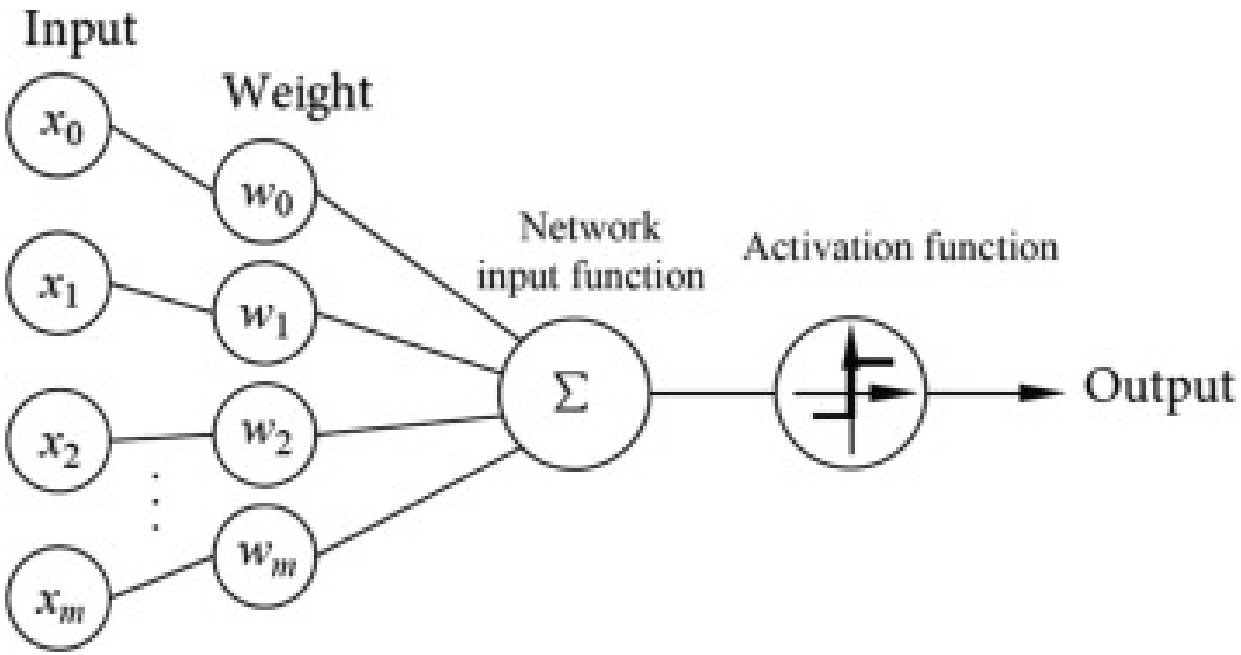
Representation Learning

Each hidden layer learns progressively abstract representations.

Activation Functions

Function	Formula
Sigmoid	$\frac{1}{1+e^{-x}}$
Tanh	$\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
ReLU	$\max(0, x)$
Leaky ReLU	$\max(0.01x, x)$
GELU	$x * \frac{1}{2}[1+\mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)]$ where $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}\,dt$

Structure of a Perceptron



- Inputs: $x_1, x_2, ..., x_n$
- Weights: $w_1, w_2, ..., w_n$
- Bias: b

Training the Perceptron

Weights are updated using the **Perceptron Learning Rule**:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

$$b \leftarrow b + \eta(y - \hat{y})$$

Where:

- η = learning rate
- y = true label
- $\hat{y}$ = predicted label

What Perceptrons Can Do

Strengths

- Solve linearly separable problems
- Perform logical operations (AND, OR)
- Form the base of Multi-Layer Perceptrons (MLPs)

What Perceptrons Cannot Do

Limitations

- Cannot solve non-linear problems like XOR
- Only outputs 0 or 1 (no probabilities)
- Limited expressive power

This leads to Multi-Layer Perceptrons and Deep Learning

Exercise : Network Specification

Consider a neural network with one hidden neuron.

- Input:

$$x = [2.0,\ -1.0]$$

- Weights and biases:

$$W_1 = [0.5 \quad -0.3], \qquad b_1 = 0.1$$

$$W_2 = 0.8, \qquad b_2 = -0.2$$

- Activation: Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Target:

$$y = 1$$

- Loss:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

- Learning rate:

$$\alpha = 0.1$$

Tasks (Forward Pass)

Compute:

1. Hidden pre-activation:
$$z_1 = W_1 x + b_1$$
2. Hidden activation:
$$a_1 = \sigma(z_1)$$
3. Output pre-activation:
$$z_2 = W_2 a_1 + b_2$$
4. Network output:
$$\hat{y} = \sigma(z_2)$$
5. Loss:
$$L = \frac{1}{2}(\hat{y} - y)^2$$

Backpropagation Tasks

1. Compute:
$$\frac{\partial L}{\partial \hat{y}}, \qquad \frac{\partial L}{\partial z_2}$$
2. Gradients:
$$\frac{\partial L}{\partial W_2}, \qquad \frac{\partial L}{\partial b_2}$$
3. Hidden layer:
$$\frac{\partial L}{\partial a_1}, \qquad \frac{\partial L}{\partial z_1}$$
4. Hidden parameters:
$$\frac{\partial L}{\partial W_1}, \qquad \frac{\partial L}{\partial b_1}$$

Gradient Descent Updates

Update the parameters:

$$W_i^{new} = W_i - \alpha \frac{\partial L}{\partial W_i}, \qquad b_i^{new} = b_i - \alpha \frac{\partial L}{\partial b_i}$$

Interpretation questions:

- What happens if W_2 is very large?
- What is the effect of very small or large learning rate?

Deep Learning Workflow

1. Data collection & preprocessing
2. Model design & hyperparameters
3. Training and validation
4. Evaluation and interpretability
5. Deployment and monitoring

Hyperparameter Tuning

- Learning rate, batch size, optimizer
- Number of layers, units per layer
- Activation and dropout rate
- Automated tuning tools: Grid search, Random search, Bayesian optimization

Gradient Descent: Objective & Update Rule

Learning Goal:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \ell(f_\theta(x^{(i)}), y^{(i)})$$

Gradient Descent Update:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$$

For a single weight w_j:

$$w_j^{(t+1)} = w_j^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial w_j}$$

Key Challenge: Neural networks are nested functions $\Rightarrow$ we must apply the **multivariable chain rule**.

Chain Rule in Deep Networks

Neural network computations:

$$x \to z^{(1)} \to a^{(1)} \to z^{(2)} \to a^{(2)} \to \hat{y}$$

Chain Rule (general form):

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

For neural networks:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

This principle extends to all layers $\Rightarrow$ **backpropagation**.

MLP Forward Pass (Two-Layer Network)

Layer 1 (Hidden Layer)

$$z^{(1)} = W^{(1)}x + b^{(1)}$$
$$a^{(1)} = \sigma(z^{(1)})$$

Layer 2 (Output Layer)

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$
$$\hat{y} = \sigma(z^{(2)})$$

Loss (MSE):

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$$

Backpropagation: Output Layer

We compute

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}}$$

Step 1: Output layer error term

$$\delta^{(2)} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} = (\hat{y} - y) \cdot \sigma'(z^{(2)})$$

Step 2: Gradients

Weights:

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \delta^{(2)} (a^{(1)})^T$$

Bias:

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)}$$

This completes the gradient for the output layer.

Backpropagation: Hidden Layer

We compute

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b^{(1)}}$$

Step 1: Hidden layer error term

$$\delta^{(1)} = \left((W^{(2)})^T \delta^{(2)} \right) \odot \sigma'(z^{(1)})$$

Step 2: Gradients

Weights:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \delta^{(1)} x^T$$

Bias:

$$\frac{\partial \mathcal{L}}{\partial b^{(1)}} = \delta^{(1)}$$

Thus each layer receives error from above and multiplies by the activation derivative.

Final Gradient Descent Updates for MLP

Output Layer Updates

$$W^{(2)} \leftarrow W^{(2)} - \eta\, \delta^{(2)} (a^{(1)})^T$$
$$b^{(2)} \leftarrow b^{(2)} - \eta\, \delta^{(2)}$$

Hidden Layer Updates

$$W^{(1)} \leftarrow W^{(1)} - \eta\, \delta^{(1)} x^T$$
$$b^{(1)} \leftarrow b^{(1)} - \eta\, \delta^{(1)}$$

Key Insight: Backpropagation = repeated application of the chain rule across layers.
Outcome: MLPs learn by propagating error backwards and adjusting weights to reduce the loss.

Multilayer Perceptron (MLP) for Classification

Key idea: MLP learns hierarchical representations and uses Softmax for multi-class outputs.

- Input: feature vector x
- Hidden layers: nonlinear transformations
- Output layer: class scores
- Softmax converts scores to probabilities
- Loss: Cross-entropy

$$\text{class}(x) = \arg\max_k \hat{y}_k$$

MLP Architecture Example

- Input dimension: 4 (Iris features)
- Hidden layer: 5 neurons
- Activation: ReLU
- Output layer: 3 neurons
- Activation: Softmax

$$x \rightarrow \text{Hidden Layer} \rightarrow \text{Output Layer} \rightarrow \hat{y}$$

Step 1: Hidden Layer Computation

$$z_1 = W_1 x + b_1$$

Activation (ReLU):

$$a_1 = \max(0, z_1)$$

ReLU introduces non-linearity.

Step 2: Output Layer Computation

$$z_2 = W_2 a_1 + b_2$$

Softmax converts scores to probabilities:

$$\hat{y}_k = \frac{e^{z_{2k}}}{\sum_j e^{z_{2j}}}$$

$$\hat{y} = [\hat{y}_1, \hat{y}_2, \hat{y}_3]$$

Step 3: Cross-Entropy Loss

True label (one-hot):

$$y = [y_1,\ y_2,\ y_3]$$

Loss:

$$L = -\sum_k y_k \log(\hat{y}_k)$$

For classification:

$$L = -\log(\hat{y}_{\text{correct class}})$$

Derivative at Output Layer

Softmax + Cross-Entropy simplifies:

$$\frac{\partial L}{\partial z_2} = \hat{y} - y$$

This gradient is used to update the output layer.

Gradients for Output Layer Parameters

$$\frac{\partial L}{\partial W_2} = (\hat{y} - y)\, a_1^T$$

$$\frac{\partial L}{\partial b_2} = \hat{y} - y$$

Gradient Flow to Hidden Layer

$$\frac{\partial L}{\partial a_1} = W_2^T(\hat{y} - y)$$

ReLU derivative:

$$\text{ReLU}'(z_1) = \begin{cases} 1 & z_1 > 0 \\ 0 & z_1 \leq 0 \end{cases}$$

Thus:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \odot \text{ReLU}'(z_1)$$

Gradients for Input Layer Parameters

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} x^T$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1}$$

Parameter Updates

Gradient descent:

$$W^{new} = W - \alpha \frac{\partial L}{\partial W}$$

$$b^{new} = b - \alpha \frac{\partial L}{\partial b}$$

Learning rate α controls the update step.

Iris Dataset (3-Class Classification)

Features:

- Sepal length
- Sepal width
- Petal length
- Petal width

Classes:

- Setosa
- Versicolor
- Virginica

MLP learns to separate these classes using learned features.

Classification Output Example

Example prediction:

$$\hat{y} = [0.95,\ 0.04,\ 0.01]$$

Meaning:

- 95% Setosa
- 4% Versicolor
- 1% Virginica

Predicted class = Setosa

Emerging Frontiers

- Self-supervised learning
- Multimodal AI (text+image+audio)
- Efficient architectures (MobileNet, TinyBERT)
- Quantum deep learning
- Civic and educational AI

Key Takeaways

- Deep learning automates hierarchical representation learning.
- Key architectures: FFNN, CNN, RNN/LSTM, Transformers, RL.
- The field blends theory, optimization, and ethics.
- Future: Explainable, energy-efficient, human-centered AI.

Mini Demo: XOR with PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim

X = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y = torch.tensor([[0.],[1.],[1.],[0.]])

model = nn.Sequential(
    nn.Linear(2, 2),
    nn.ReLU(),
    nn.Linear(2, 1),
    nn.Sigmoid()
)

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.1)

for epoch in range(2000):
    optimizer.zero_grad()
    y_pred = model(X)
    loss = criterion(y_pred, y)
    loss.backward()
    optimizer.step()

print(model(X))
```

Keras Example

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

model = Sequential([
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val),
          epochs=25, batch_size=32)
```

Exercise 1: Forward Pass Computation

Given: A 1-hidden-layer MLP

$$W_1 = \begin{bmatrix} 0.3 & -0.2 & 0.1 \\ 0.5 & 0.4 & -0.3 \end{bmatrix}, \quad b_1 = [0.1,\ -0.05]$$

$$W_2 = [0.4,\ -0.1], \quad b_2 = 0.2$$

Input:

$$x = [1.0,\ 0.5,\ -1.0]$$

Tasks:

1. Compute $z_1 = W_1 x + b_1$
2. Apply ReLU activation
3. Compute output $z_2 = W_2 a_1 + b_2$
4. Apply Sigmoid to get $\hat{y}$

Exercise 2: Backpropagation

Using the same MLP:

$$L = \frac{1}{2}(\hat{y} - y)^2, \qquad y = 1$$

Tasks:

1. Compute
$$\frac{\partial L}{\partial \hat{y}} \qquad \frac{\partial L}{\partial z_2}$$
2. Compute gradients:
$$\frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2}$$
3. Backpropagate to hidden layer:
$$\frac{\partial L}{\partial z_1}$$
4. Compute:
$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}$$

Exercise 3: Model Comparison

Compare **Logistic Regression**, **SVM**, and **MLP** for the Iris dataset.
Tasks:

1. Train all three models on the same train/test split.
2. Report:
   - Accuracy
   - Precision/Recall/F1
   - Confusion Matrix
3. Discuss:
   - Which model performed best?
   - How does dataset size affect each model?
   - What about interpretability vs predictive power?

Exercise 4: Practical Implementation

Implement in Python (PyTorch / Scikit-Learn):

1. MLP classifier for Iris (1 hidden layer)
2. Logistic regression classifier
3. SVM classifier (linear + RBF kernel)

Tasks:

- Compare training time
- Plot decision boundaries for 2 features
- Analyze misclassified samples

Challenge Exercise

Build a deeper MLP:

$$4 \rightarrow 10 \rightarrow 5 \rightarrow 3$$

Tasks:

1. Derive gradients for all layers
2. Train it on the Iris dataset
3. Compare performance to a shallow MLP

Goal: Understand the effect of depth on classification.

Thank You!

Questions and Group Discussion

Recommended Reading

- Goodfellow, Bengio, Courville (2016). *Deep Learning*.
- Chollet (2021). *Deep Learning with Python*.
- Bishop (2006). *Pattern Recognition and Machine Learning*.
- Online: TensorFlow, PyTorch.

References

Fleuret, F. (2024).
The Little Book of Deep Learning.
University of Geneva.

Goodfellow, I., Bengio, Y., Courville, A. (2016).
Deep Learning.
MIT Press, Cambridge.

Bishop, C. M., & Bishop, H. (2023).
Deep Learning: Foundations and Concepts.
Springer Nature.

Prince, S. J. (2023).
Understanding Deep Learning.
MIT Press.

Kelleher, J. D. (2019).
Deep Learning.
MIT Press.